

Author: Collin Aslin Dsouza

```
In [1]: # Core libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-learn modules
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import mean_absolute_error, r2_score, mean_squared_error
```

```
In [2]: #Load the dataset
kpi = pd.read_excel("Supplier_KPI.xlsx")
esg = pd.read_excel("Supplier_ESG.xlsx")
```

```
In [3]: #Merge Dataset
df = pd.merge(kpi, esg, how='inner', on='Supplier_ID')
```

```
In [4]: #Check for missing values and data type
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 19 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Supplier_ID                          200 non-null    object
 1   On_Time_Delivery_Rate                200 non-null    float64
 2   Lead_Time_Days                      200 non-null    int64
 3   Order_Accuracy                      200 non-null    float64
 4   Defect_Rate                         200 non-null    float64
 5   Cost_Competitiveness_Index          200 non-null    float64
 6   Capacity_Utilization                200 non-null    float64
 7   Flexibility_Score                   200 non-null    int64
 8   Financial_Stability_Score           200 non-null    int64
 9   Dependency_Ratio                    200 non-null    float64
10   Carbon_Emissions                    200 non-null    float64
11   Energy_Efficiency                   200 non-null    float64
12   Waste_Management_Score              200 non-null    int64
13   Water_Usage                        200 non-null    float64
14   Labor_Practices_Score               200 non-null    int64
15   Health_Safety_Incidents             200 non-null    int64
16   Community_Engagement_Index          200 non-null    int64
17   Governance_Transparency_Score       200 non-null    int64
18   ESG_Certification_Status            200 non-null    int64
dtypes: float64(9), int64(9), object(1)
memory usage: 29.8+ KB
```

All 19 columns have 200 non-null entries, confirming a clean dataset with no missing data.

All the KPI and ESG metrics are in a numeric format.

```
In [5]: #Summary Statistics
df.describe().T
```

Out[5]:

	count	mean	std	min	25%	50%	75%	max
On_Time_Delivery_Rate	200.0	85.28530	8.702502	70.10	77.9925	85.150	92.4050	99.89
Lead_Time_Days	200.0	29.80500	17.025461	3.00	15.0000	29.000	46.0000	59.00
Order_Accuracy	200.0	92.67715	4.291209	85.05	88.9000	93.175	96.1275	99.93
Defect_Rate	200.0	5.20615	2.919622	0.01	2.6450	5.630	7.7350	10.00
Cost_Competitiveness_Index	200.0	1.00730	0.111971	0.80	0.9200	1.010	1.1025	1.20
Capacity_Utilization	200.0	75.92565	14.258323	50.08	63.9250	76.400	87.5275	99.85
Flexibility_Score	200.0	5.44000	2.871870	1.00	3.0000	5.000	8.0000	10.00
Financial_Stability_Score	200.0	54.29500	27.284799	1.00	32.7500	58.500	77.0000	100.00
Dependency_Ratio	200.0	20.03670	11.275152	1.22	10.6450	19.590	30.7725	39.56
Carbon_Emissions	200.0	2.42175	1.454336	0.19	1.0900	2.310	3.7775	4.99
Energy_Efficiency	200.0	58.53330	24.075494	20.04	36.4175	58.185	81.3775	99.95
Waste_Management_Score	200.0	5.46500	2.824654	1.00	3.0000	6.000	8.0000	10.00
Water_Usage	200.0	264.35155	132.212743	12.62	161.3675	280.105	377.6725	483.25
Labor_Practices_Score	200.0	5.48500	2.717850	1.00	3.0000	5.500	8.0000	10.00
Health_Safety_Incidents	200.0	10.97500	6.232060	0.00	6.0000	12.000	16.0000	20.00
Community_Engagement_Index	200.0	5.33500	2.906028	1.00	3.0000	5.000	8.0000	10.00
Governance_Transparency_Score	200.0	5.60000	2.776426	1.00	3.0000	6.000	8.0000	10.00
ESG_Certification_Status	200.0	0.49000	0.501154	0.00	0.0000	0.000	1.0000	1.00

Summary Statistics

Operational Performance:

On-time delivery (mean 85%) and order accuracy (93%) are strong, though lead times vary widely (3–59 days), suggesting room for consistency improvement.

⚙️ Quality & Efficiency:

Defect rates average ~5%, with capacity utilization around 76%—healthy but with potential to optimize further.

Sustainability:

Carbon emissions and energy efficiency show wide variation, highlighting opportunities for greener practices. ESG certification is below 50%, suggesting limited adoption.

Social & Governance:

Labor practices, community engagement, and transparency scores cluster around mid-levels (~5–6/10), pointing to moderate but improvable ESG maturity.

Need for Scaling:

The extreme difference in scale confirms the absolute necessity of the subsequent Standard Scaling step. If unscaled, features like Water_Usage would dominate the Euclidean distance calculation in the K-Means algorithm, effectively rendering the lower-magnitude scores (like Defect_Rate or any of the scores) irrelevant in forming clusters.

In [6]:

```
# Keep Supplier_ID aside for later mapping
supplier_ids = df["Supplier_ID"]

# Drop Supplier_ID since it's categorical
numeric_df = df.drop(columns=["Supplier_ID"])
```

- Supplier_ID is stored separately since it's non-numeric and not usable for clustering, but essential for mapping clusters back to suppliers later.
- numeric_df contains only quantitative KPI and ESG features, which serve as the input for clustering.

In [7]:

```
#Scaling the dataset
from sklearn.preprocessing import StandardScaler

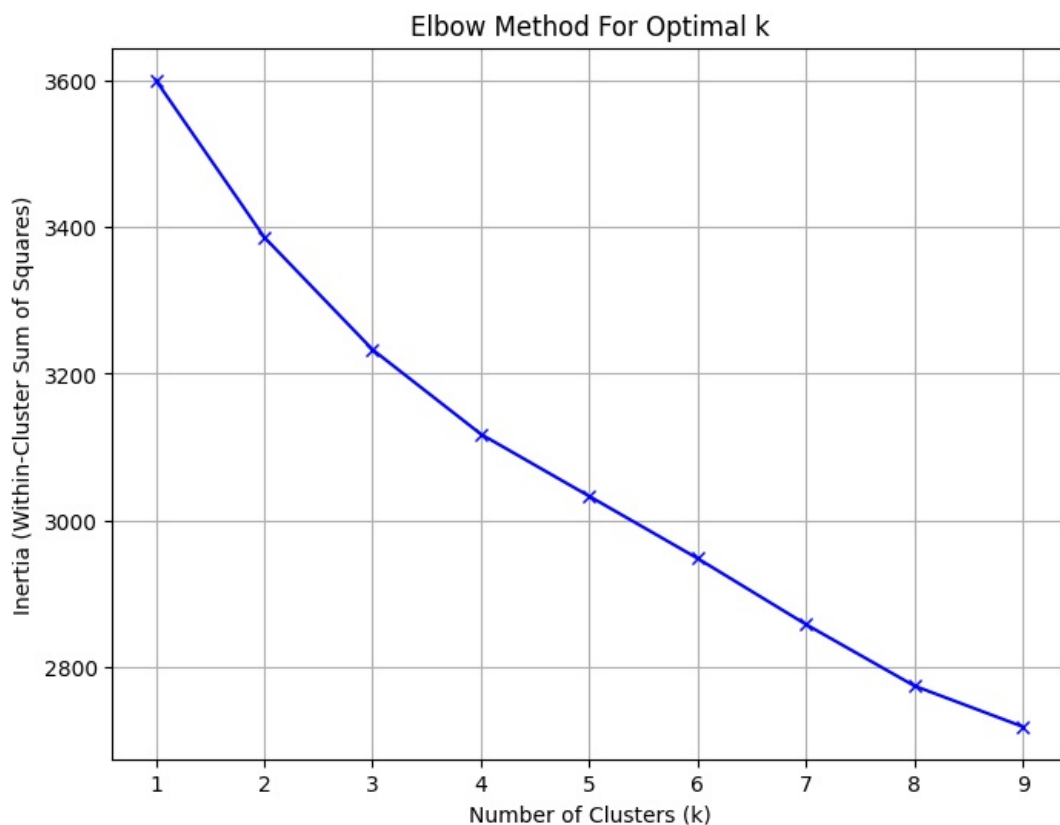
scaler = StandardScaler()
scaled_features = scaler.fit_transform(numeric_df)
```

```
In [8]: # The Elbow Method: Calculate Inertia for k=1 to k=9

inertia = []
K = range(1, 10) # Test k from 1 to 9

for k in K:
    # Initialize and fit the model for each k
    kmeans_model = KMeans(n_clusters=k, random_state=123, n_init=25)
    kmeans_model.fit(scaled_features)
    # Append the inertia (WSS) to the list
    inertia.append(kmeans_model.inertia_)

# Plot the Elbow Curve
plt.figure(figsize=(8, 6))
plt.plot(K, inertia, 'bx-')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia (Within-Cluster Sum of Squares)')
plt.title('Elbow Method For Optimal k')
plt.grid(True)
plt.show()
```



```
In [9]: # build the k means model
kmeans = KMeans(n_clusters=4, random_state=123, n_init=25)
kmeans.fit(scaled_features)
```

```
Out[9]: KMeans
KMeans(n_clusters=4, n_init=25, random_state=123)
```

```
In [10]: # view the clustering result
print("Cluster center:\n", kmeans.cluster_centers_)
print("Cluster labels:\n", kmeans.labels_)
print("Inertia(distortion):\n", kmeans.inertia_)
```

```

Cluster center:
[[ 0.24556454 -0.01453587 -0.28961981 -0.30204109  0.7050438  0.30906693
  0.12242118  0.55482328 -0.1544982  0.76057763  0.21877406 -0.68502548
  0.33433275  0.06986917  0.46042431 -0.04336266  0.12763725  0.04326447]
[-0.14002443 -0.65376992  0.19337693  0.26980779  0.19943075  0.14978292
  0.17320258 -0.24614682  0.42019054  0.12125881 -0.27511363  0.34090632
  0.45580906 -0.40649289 -0.87901502  0.54502507 -0.46248851 -0.17152367]
[ 0.25161651 -0.41793254  0.23781795 -0.51783613 -0.06972682 -0.55008796
 -0.51970193  0.12089599 -0.76581274 -0.17874946 -0.01737241  0.27644346
 -0.80866086 -0.17889768  0.62785714  0.02747185 -0.21664744 -0.10197162]
[-0.20716527  0.70271686 -0.09254492  0.31214612 -0.53378737  0.03223088
  0.11453837 -0.24993096  0.26511289 -0.4503663  0.06138152  0.03042488
 -0.0383223  0.33964633 -0.06125626 -0.36054971  0.36421889  0.15046488]]
Cluster labels:
[0 2 2 1 1 1 2 3 2 1 3 1 2 1 0 1 1 3 1 2 2 2 3 3 2 3 1 1 3 1 3 1 0 2 2 3 1
 1 2 1 3 0 3 0 0 1 3 2 3 1 1 2 3 3 1 3 2 0 2 3 3 0 0 1 3 1 2 3 2 3 3 1 3 3
 3 1 3 0 0 3 3 2 2 1 1 0 3 3 3 1 1 2 1 3 3 2 0 0 1 2 1 3 2 0 0 2 1 3 3 3 1
 3 2 3 1 0 0 0 3 1 3 3 0 1 1 0 0 3 2 2 0 3 0 0 1 3 3 0 1 0 0 3 3 0 3 2 3 3
 1 3 0 0 0 3 2 2 1 3 2 3 0 1 3 0 0 3 1 2 2 2 2 2 1 1 0 2 0 3 3 0 2 0 3 3 1
 2 2 0 3 3 3 0 0 3 3 0 3 1 3 3]
Inertia(distortion):
3117.7644399455453

```

Cluster Centers: These are the mean values of each feature (in scaled form) for each of the four clusters. They represent the "prototypical" supplier in each cluster.

Cluster Labels: This is an array assigning each of the 200 suppliers (by index) to one of the four clusters (0, 1, 2, or 3). Inertia: The Within-Cluster Sum of Squares (WSS) is 3117.76. This value measures how cohesive the clusters are (lower is better). While this number alone isn't highly informative, it is used in the next step to assess the model fit.

```

In [11]: df["Cluster_Euclidean"] = kmeans.labels_
print(df[["Supplier_ID", "Cluster_Euclidean"]].head())

Supplier_ID  Cluster_Euclidean
0      SUP0001                0
1      SUP0002                2
2      SUP0003                2
3      SUP0004                1
4      SUP0005                1

```

The calculated cluster labels (`kmeans.labels_`) are now added as a new column, `Cluster_Euclidean`, to the original dataframe (`df`).

```

In [12]: # CLUSTER SIZE DISTRIBUTION
plt.figure(figsize=(6, 6))
sns.countplot(
    data=df,
    x='Cluster_Euclidean',
    palette='viridis'
)

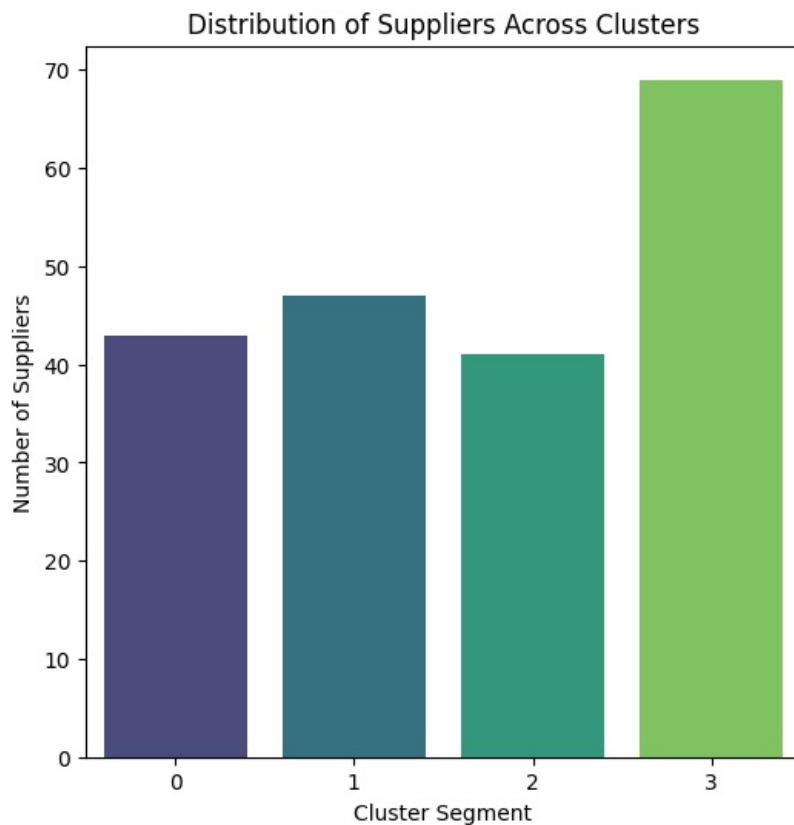
plt.title('Distribution of Suppliers Across Clusters')
plt.xlabel('Cluster Segment')
plt.ylabel('Number of Suppliers')
plt.show()

```

C:\Users\colli\AppData\Local\Temp\ipykernel_9068\2351572079.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(
```



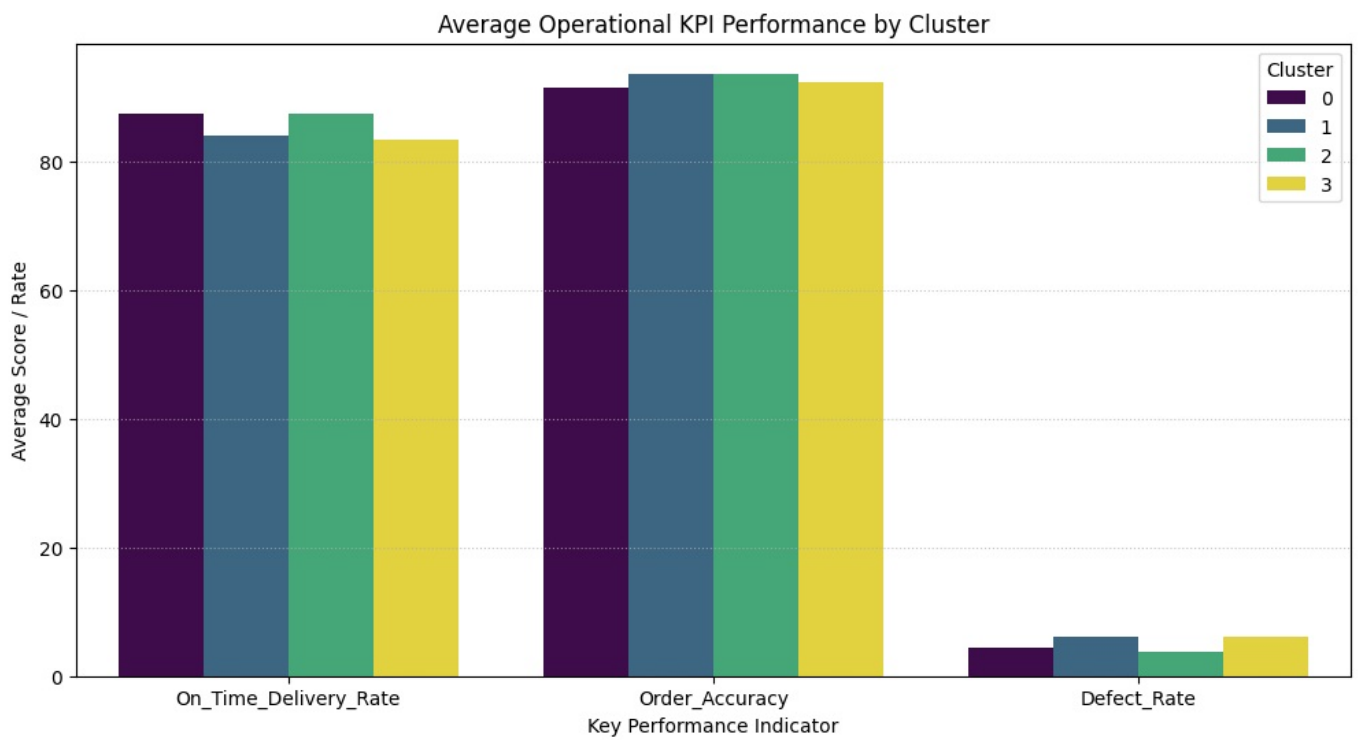
```
In [13]: # 2. BAR CHART: AVERAGE OPERATIONAL KPIS BY CLUSTER
kpi_columns = ['On_Time_Delivery_Rate', 'Order_Accuracy', 'Defect_Rate']

# Calculate the mean values for the visualization
kpi_summary = df.groupby('Cluster_Euclidean')[kpi_columns].mean().reset_index()
kpi_summary = pd.melt(kpi_summary, id_vars='Cluster_Euclidean', var_name='KPI', value_name='Average Value')

plt.figure(figsize=(12, 6))
sns.barplot(
    data=kpi_summary,
    x='KPI',
    y='Average Value',
    hue='Cluster_Euclidean',
    palette='viridis'
)

plt.title('Average Operational KPI Performance by Cluster')
plt.ylabel('Average Score / Rate')
plt.xlabel('Key Performance Indicator')

plt.legend(title='Cluster')
plt.grid(axis='y', linestyle=':', alpha=0.7)
plt.show()
```

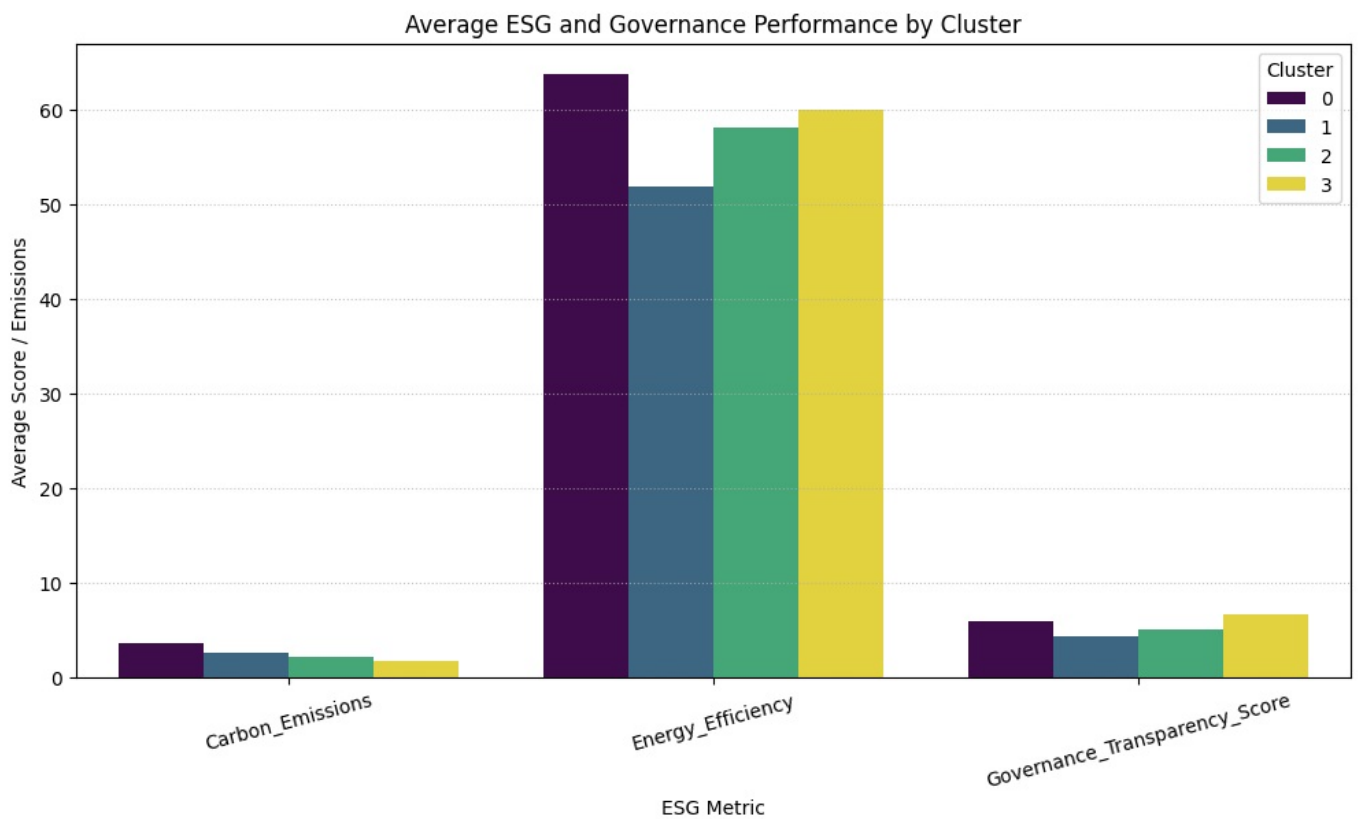


```
In [19]: # 3. BAR CHART: AVERAGE ESG METRICS BY CLUSTER
esg_columns = ['Carbon_Emissions', 'Energy_Efficiency', 'Governance_Transparency_Score']

# Calculate the mean values for the visualization
esg_summary = df.groupby('Cluster_Euclidean')[esg_columns].mean().reset_index()
esg_summary = pd.melt(esg_summary, id_vars='Cluster_Euclidean', var_name='ESG Metric', value_name='Average Value')

plt.figure(figsize=(12, 6))
sns.barplot(
    data=esg_summary,
    x='ESG Metric',
    y='Average Value',
    hue='Cluster_Euclidean',
    palette='viridis'
)

plt.title('Average ESG and Governance Performance by Cluster')
plt.ylabel('Average Score / Emissions')
plt.xlabel('ESG Metric')
plt.xticks(rotation=15)
plt.legend(title='Cluster')
plt.grid(axis='y', linestyle=':', alpha=0.7)
plt.show()
```



```
In [14]: # Look at average KPI/ESG values per cluster
cluster_summary = df.groupby("Cluster_Euclidean")[["On_Time_Delivery_Rate",
                                                    "Lead_Time_Days",
                                                    "Order_Accuracy",
                                                    "Energy_Efficiency",
                                                    "Carbon_Emissions"]].mean()

print(cluster_summary)
```

Cluster_Euclidean	On_Time_Delivery_Rate	Lead_Time_Days	Order_Accuracy \
0	87.416977	29.558140	91.437442
1	84.069787	18.702128	93.504894
2	87.469512	22.707317	93.695122
3	83.486957	41.739130	92.281014

Cluster_Euclidean	Energy_Efficiency	Carbon_Emissions
0	63.787209	3.525116
1	51.926383	2.597660
2	58.116098	2.162439
3	60.007391	1.768406

Cluster 0 – High-Impact Performers

- Delivery: Highest on-time (87.4%)
- ☐ Lead Time: Mid-range (29.6 days)
- ✔ Accuracy: Lowest (91.4%)
- ⚡ Efficiency: Highest (63.8%)
- Carbon: Highest emissions (3.53)
- Interpretation: Strong operational efficiency but heavy environmental footprint.
- Action: Build strategic partnerships, mitigate risks, and prioritize decarbonization initiatives.

⚡ Cluster 1 – Agile but Underdeveloped

- Delivery: Low (84.1%)
- ☐ Lead Time: Fastest (18.7 days)
- ✔ Accuracy: High (93.5%)
- ⚡ Efficiency: Lowest (51.9%)
- Carbon: Mid-level emissions (2.60)
- Interpretation: Quick and accurate, but lacking maturity in efficiency and sustainability.
- Action: Invest in capability building and conduct audits to strengthen delivery & ESG metrics.

Cluster 2 – Preferred / Balanced

- Delivery: Highest (87.5%)
- □ Lead Time: Low (22.7 days)
- ✓ Accuracy: Highest (93.7%)
- ✗ Efficiency: Mid (58.1%)
- Carbon: Mid (2.16)
- Interpretation: Best overall balance—strong delivery, accuracy, and reasonable ESG.
- Action: Treat as Tier 1 suppliers, integrate deeply, and drive operational excellence.

Cluster 3 – High-Risk / Sustainable Niche

- Delivery: Lowest (83.5%)
- □ Lead Time: Longest (41.7 days)
- ✓ Accuracy: Mid (92.3%)
- ✗ Efficiency: Mid (60.0%)
- Carbon: Lowest emissions (1.77)
- Interpretation: Environmentally strong but operationally weak.
- Action: Use selectively for non-urgent, sustainability-focused projects, or transition out if performance gaps are critical.

```
In [15]: import numpy as np
# Total Sum of Squares (TSS)
tss = np.sum(np.var(scaled_features, axis=0) * len(scaled_features))

# Within-Cluster Sum of Squares (WSS) = Inertia
wss = kmeans.inertia_

# Between-Cluster Sum of Squares (BSS)
bss = tss - wss

# Explained Variance (%)
explained_variance = (bss / tss) * 100
print(f"Explained Variance: {explained_variance:.2f}%")

### Above 70% is considered good
```

Explained Variance: 13.40%

Clustering Quality Interpretation

- TSS (Total Sum of Squares): Captures the overall variation in the dataset.
- BSS (Between-Cluster Sum of Squares): Reflects the variation explained by the clustering, i.e., separation between cluster centers.
- Explained Variance (BSS/TSS): At 13.40%, the clusters explain only a modest portion of the total variation.

Insights

- A low explained variance indicates that the clusters are not sharply separated, but it does not invalidate the model.
- Clustering often reveals practical groupings even when statistical separation is weak—especially in complex, multi-dimensional ESG and KPI data.
- The clusters can still provide actionable insights if interpreted carefully, e.g., identifying supplier segments with distinct trade-offs (speed vs sustainability, accuracy vs emissions).
- Instead of discarding the model, the results should be used as a strategic lens: guiding partnerships, investments, and risk mitigation while acknowledging overlap between groups.

💡 In short: 13.40% variance is low, but the clusters remain valuable as directional insights rather than rigid boundaries.

Analysis

Clustering suppliers using both operational KPIs and ESG metrics is a modern analytical approach that directly addresses today's business challenges. In a globalized economy, companies face increasing pressure to balance efficiency, resilience, and sustainability. This type of modelling is valuable because:

- Risk Management:

It identifies high-risk suppliers (e.g., poor delivery but strong ESG) versus reliable partners, enabling proactive mitigation.

- Strategic Segmentation:

Firms can classify suppliers into tiers (e.g., agile, balanced, sustainable niche) and tailor engagement strategies accordingly.

- Sustainability Integration:

ESG clustering ensures that environmental and social responsibility are embedded into procurement decisions, not treated as afterthoughts.

- Operational Excellence:

By grouping suppliers with similar performance profiles, companies can streamline negotiations, benchmarking, and improvement programs.

- Resilience & Adaptability:

Clusters highlight trade-offs (speed vs. sustainability, cost vs. quality), helping businesses design flexible supply chain strategies that withstand disruptions.

Strategic Use in Supply Chain Deployment

- Tiered Partnerships:

Prioritize “balanced” clusters for core operations, while leveraging “sustainable niche” suppliers for ESG-focused projects.

- Targeted Investments:

Direct capability-building programs to underdeveloped clusters, raising overall supply chain maturity.

- Policy Alignment:

Align procurement with corporate sustainability goals and regulatory requirements.

- Competitive Advantage:

Companies that integrate ESG into supplier strategy gain reputational benefits, investor confidence, and long-term resilience.

In short: This modelling transforms raw supplier data into actionable strategy, enabling businesses to build supply chains that are efficient, sustainable, and future-ready.

In []:

Processing math: 100%